

Homework 6

Joseph Shin

2025-09-27

Question 9.1

Using the same crime data set `uscrime.txt` as in Question 8.2, apply Principal Component Analysis and then create a regression model using the first few principal components. Specify your new model in terms of the original variables (not the principal components), and compare its quality to that of your solution to Question 8.2. You can use the R function `prcomp` for PCA. (Note that to first scale the data, you can include `scale. = TRUE` to scale as part of the PCA function. Don't forget that, to make a prediction for the new city, you'll need to unscale the coefficients (i.e., do the scaling calculation in reverse)!)

```
crime = read.table('uscrime.txt', header = TRUE)
crime = crime[-c(4, 26),]
crime
```

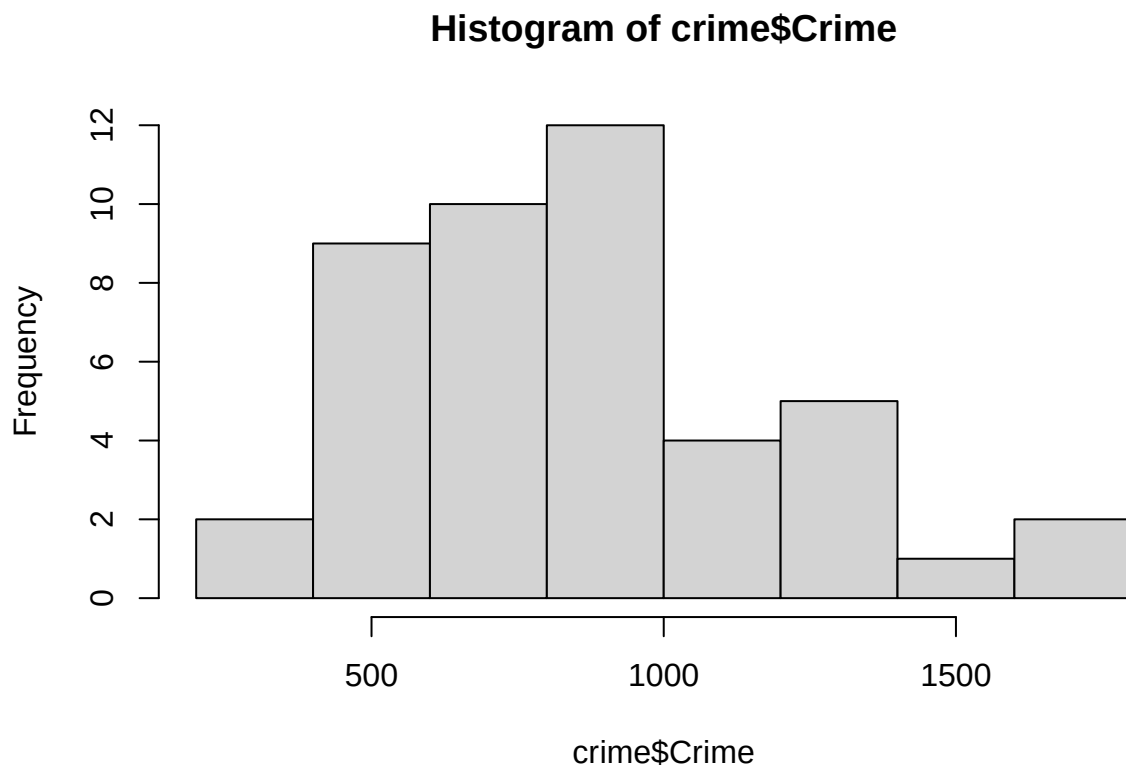
##		M	So	Ed	Po1	Po2	LF	M.F	Pop	NW	U1	U2	Wealth	Ineq	Prob
## 1	15.1	1	9.1	5.8	5.6	0.510	95.0	33	30.1	0.108	4.1	3940	26.1	0.084602	
## 2	14.3	0	11.3	10.3	9.5	0.583	101.2	13	10.2	0.096	3.6	5570	19.4	0.029599	
## 3	14.2	1	8.9	4.5	4.4	0.533	96.9	18	21.9	0.094	3.3	3180	25.0	0.083401	
## 5	14.1	0	12.1	10.9	10.1	0.591	98.5	18	3.0	0.091	2.0	5780	17.4	0.041399	
## 6	12.1	0	11.0	11.8	11.5	0.547	96.4	25	4.4	0.084	2.9	6890	12.6	0.034201	
## 7	12.7	1	11.1	8.2	7.9	0.519	98.2	4	13.9	0.097	3.8	6200	16.8	0.042100	
## 8	13.1	1	10.9	11.5	10.9	0.542	96.9	50	17.9	0.079	3.5	4720	20.6	0.040099	
## 9	15.7	1	9.0	6.5	6.2	0.553	95.5	39	28.6	0.081	2.8	4210	23.9	0.071697	
## 10	14.0	0	11.8	7.1	6.8	0.632	102.9	7	1.5	0.100	2.4	5260	17.4	0.044498	
## 11	12.4	0	10.5	12.1	11.6	0.580	96.6	101	10.6	0.077	3.5	6570	17.0	0.016201	
## 12	13.4	0	10.8	7.5	7.1	0.595	97.2	47	5.9	0.083	3.1	5800	17.2	0.031201	
## 13	12.8	0	11.3	6.7	6.0	0.624	97.2	28	1.0	0.077	2.5	5070	20.6	0.045302	
## 14	13.5	0	11.7	6.2	6.1	0.595	98.6	22	4.6	0.077	2.7	5290	19.0	0.053200	
## 15	15.2	1	8.7	5.7	5.3	0.530	98.6	30	7.2	0.092	4.3	4050	26.4	0.069100	
## 16	14.2	1	8.8	8.1	7.7	0.497	95.6	33	32.1	0.116	4.7	4270	24.7	0.052099	
## 17	14.3	0	11.0	6.6	6.3	0.537	97.7	10	0.6	0.114	3.5	4870	16.6	0.076299	
## 18	13.5	1	10.4	12.3	11.5	0.537	97.8	31	17.0	0.089	3.4	6310	16.5	0.119804	
## 19	13.0	0	11.6	12.8	12.8	0.536	93.4	51	2.4	0.078	3.4	6270	13.5	0.019099	
## 20	12.5	0	10.8	11.3	10.5	0.567	98.5	78	9.4	0.130	5.8	6260	16.6	0.034801	
## 21	12.6	0	10.8	7.4	6.7	0.602	98.4	34	1.2	0.102	3.3	5570	19.5	0.022800	
## 22	15.7	1	8.9	4.7	4.4	0.512	96.2	22	42.3	0.097	3.4	2880	27.6	0.089502	
## 23	13.2	0	9.6	8.7	8.3	0.564	95.3	43	9.2	0.083	3.2	5130	22.7	0.030700	
## 24	13.1	0	11.6	7.8	7.3	0.574	103.8	7	3.6	0.142	4.2	5400	17.6	0.041598	
## 25	13.0	0	11.6	6.3	5.7	0.641	98.4	14	2.6	0.070	2.1	4860	19.6	0.069197	
## 27	13.5	0	10.9	6.9	7.1	0.540	96.5	6	0.4	0.080	2.2	5640	13.9	0.036099	
## 28	15.2	0	11.2	8.2	7.6	0.571	101.8	10	7.9	0.103	2.8	5370	21.5	0.038201	
## 29	11.9	0	10.7	16.6	15.7	0.521	93.8	168	8.9	0.092	3.6	6370	15.4	0.023400	
## 30	16.6	1	8.9	5.8	5.4	0.521	97.3	46	25.4	0.072	2.6	3960	23.7	0.075298	

[illegible]

```
## 39 27.3004 826
## 40 29.3004 1151
## 41 30.0001 880
## 42 12.1996 542
## 43 31.9989 823
## 44 30.0001 1030
## 45 32.5996 455
## 46 16.6999 508
## 47 16.0997 849
```

First, I removed the outliers from the original dataset where the crime rate was valued at 1993 and 1969. The reason is that Principal Component Analysis can be sensitive to outliers.

```
hist(crime$Crime)
```



From this histogram, we see that for lower crime rates, there is less variance, while for higher crime rates, there is higher variance. Therefore, we know that heteroscedasticity exists in the data. I will perform Box Cox transformation to equalize the variance.

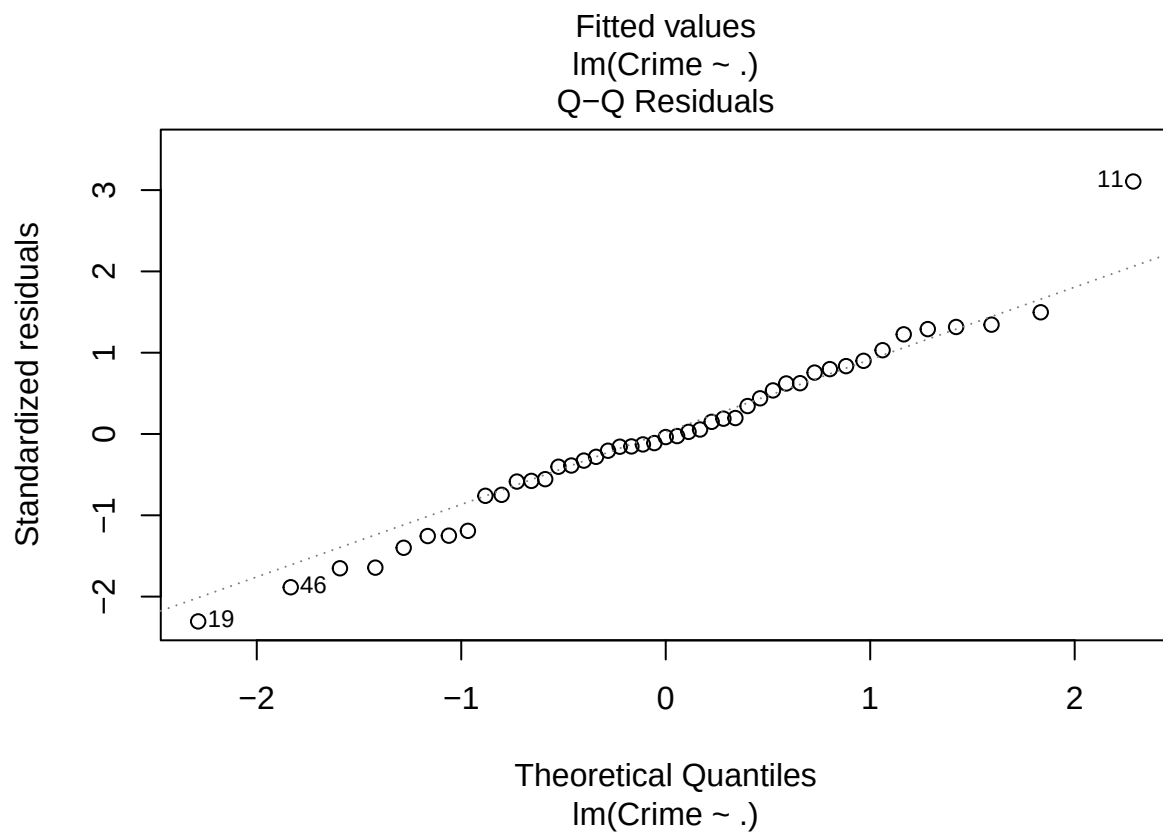
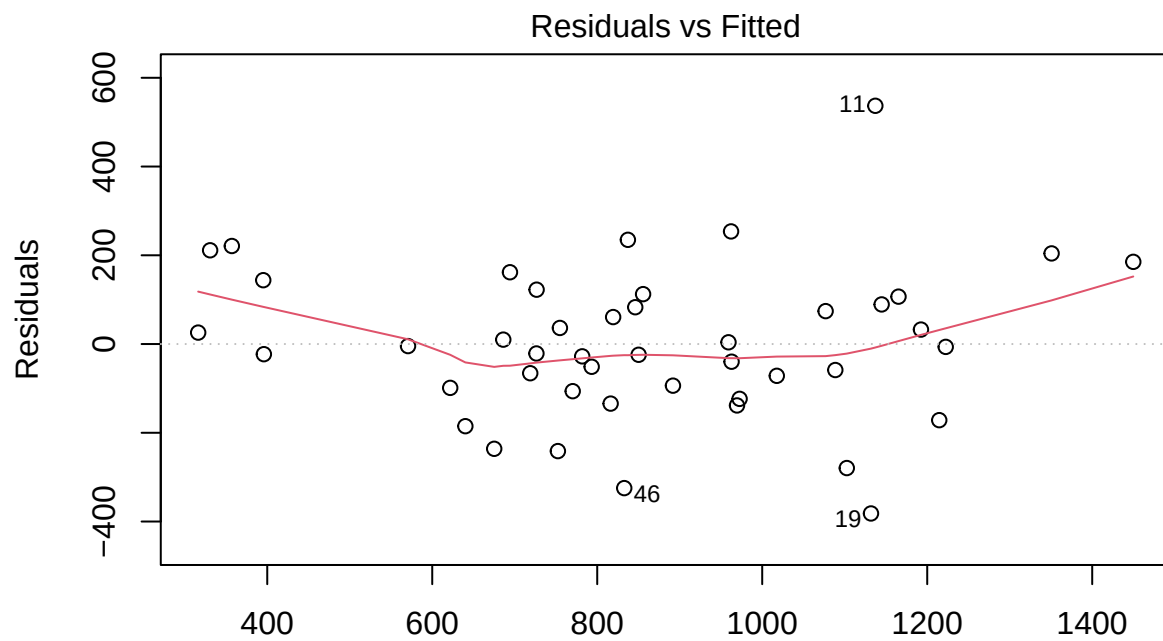
```
library(MASS)
library(car)
```

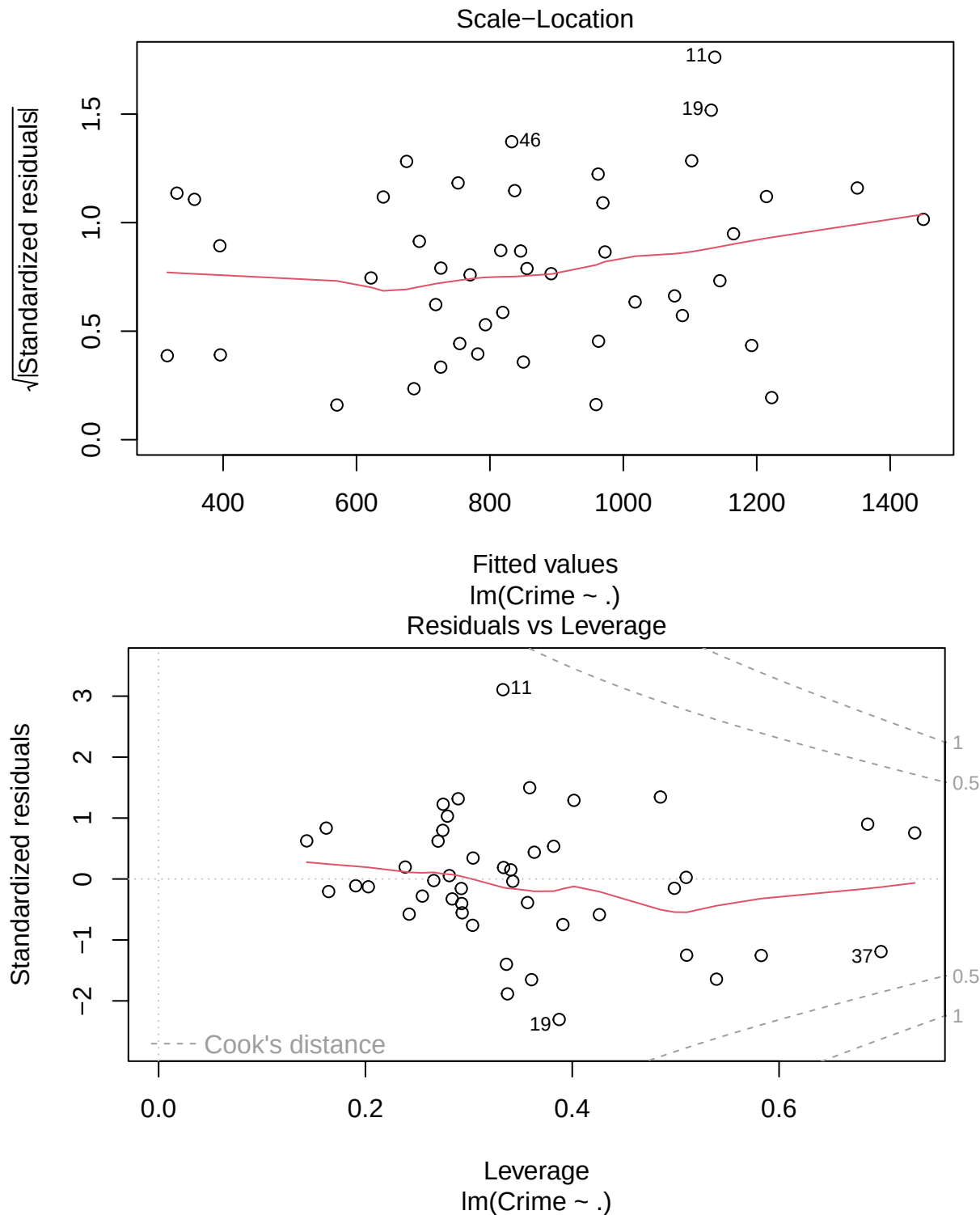
```
## Warning: package 'car' was built under R version 4.3.3
```

```
## Loading required package: carData
```

To start off, I created my first Principal Component Analysis function with the data and scaled it.

```
model = lm(Crime ~., data = crime)
plot(model)
```





On the residual plot, the residuals fan out, forming a “U” shape, or a funnel shape. This means that heteroscedasticity (unequal variance) is present in this model. The lower and higher fitted values have a higher variance while the values in between have a lower variance. We should transform the response variable, Crime, to equal out the variances. We will equal the variance through PCA.

```
pca = prcomp(crime[,1:15], scale. = TRUE)
summary(pca)
```

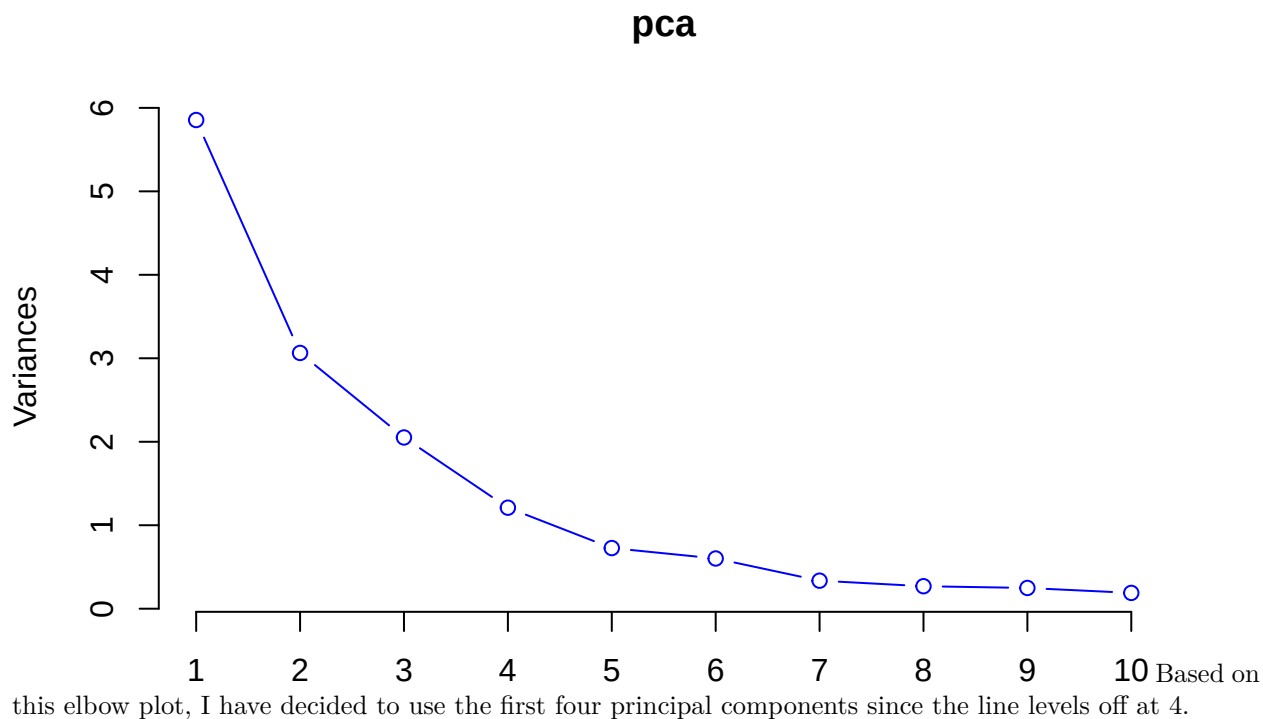
```
## Importance of components:
##              PC1    PC2    PC3    PC4    PC5    PC6    PC7
## Standard deviation  2.4196 1.7504 1.4322 1.10006 0.85240 0.7756 0.57949
## Proportion of Variance 0.3903 0.2043 0.1368 0.08068 0.04844 0.0401 0.02239
## Cumulative Proportion 0.3903 0.5946 0.7313 0.81199 0.86043 0.9005 0.92292
##              PC8    PC9    PC10    PC11    PC12    PC13    PC14
## Standard deviation  0.51908 0.49851 0.43510 0.42217 0.36402 0.26808 0.24737
## Proportion of Variance 0.01796 0.01657 0.01262 0.01188 0.00883 0.00479 0.00408
## Cumulative Proportion 0.94088 0.95745 0.97007 0.98195 0.99078 0.99558 0.99966
##              PC15
## Standard deviation  0.07191
## Proportion of Variance 0.00034
## Cumulative Proportion 1.00000
```

`pca$rotation` is the matrix of the eigenvectors.

```
pca$rotation[,1:4]
```

```
##              PC1    PC2    PC3    PC4
## M      -0.316254616  0.06305042  0.188947996 -0.08590892
## So     -0.333187182 -0.17274096  0.033066443  0.23635812
## Ed      0.333128872  0.23511610  0.067982291  0.08785745
## Po1     0.306499286 -0.29432187  0.059628179  0.24644907
## Po2     0.308676618 -0.28004655  0.058722722  0.27659132
## LF      0.154517412  0.31717488  0.296573574 -0.23806501
## M.F     0.064689141  0.42952225 -0.187822287 -0.11839142
## Pop     0.109365270 -0.44953772  0.050640418 -0.09615164
## NW     -0.304294463 -0.23263251  0.097529594  0.11826877
## U1      0.022414151  0.03915007 -0.656623612 -0.16953306
## U2     -0.005556309 -0.25276966 -0.582841039 -0.11135476
## Wealth  0.383828141 -0.06408123 -0.003046747  0.10176965
## Ineq   -0.374247145 -0.02940729  0.014667912 -0.12634975
## Prob   -0.260948770  0.11704589 -0.086774938  0.52844557
## Time   -0.011946310 -0.34708314  0.194055820 -0.59258263
```

```
screeplot(pca, type = 'lines', col = 'blue')
```



Get first 4 principal components.

```
pc = pca$x[,1:4]
pc
```

##	PC1	PC2	PC3	PC4
## 1	-4.0935023	-1.20791422	-1.01576665	0.65161544
## 2	1.3267560	0.74207890	-0.05059019	-0.27276984
## 3	-4.1109757	0.20258827	-0.27996077	0.48556231
## 5	2.1027863	1.21385525	1.29641877	0.84722764
## 6	3.2732963	-0.53145666	0.46153018	1.55935373
## 7	0.4426226	-0.07057371	-0.91765719	1.30111913
## 8	0.1564587	-1.59102807	0.62074390	1.28712204
## 9	-3.4407841	-0.86419462	1.35590684	0.37604683
## 10	1.2241704	3.17748101	0.41712755	-0.29216490
## 11	2.9087799	-2.80524191	1.36788138	-1.11494471
## 12	1.2158063	-0.03221376	1.07898532	-1.24214776
## 13	0.6952647	1.01415029	1.75896396	-1.42222275
## 14	0.5682556	1.86328628	1.01678308	0.14663130
## 15	-3.2902654	0.02214333	-0.95406889	0.09512920
## 16	-2.8857297	-2.19469140	-1.76700390	0.38840616
## 17	-0.1108991	1.45014318	-1.34747964	0.78984391
## 18	0.1853067	-0.76924601	-0.24176781	3.99787914
## 19	3.3528201	-1.87087067	0.87329731	1.10675871
## 20	2.4748383	-1.70934503	-2.86311368	-0.37400918
## 21	1.3066347	0.28485024	0.16940801	-2.11854103
## 22	-5.5896782	-1.13701468	0.19088914	-0.28921523
## 23	0.1425704	-0.72542666	0.66709119	-0.28592178

```
## 24 1.3793789 2.21833180 -2.94849086 -0.31291788
## 25 0.4079536 2.58581260 1.86882341 0.28767014
## 27 1.1470342 0.77583064 1.14904398 -0.06055945
## 28 0.2572471 1.57195524 0.11269182 -0.60785743
## 29 4.0979836 -5.20884798 0.57507470 0.52075535
## 30 -3.9722150 -0.53307313 1.47988572 0.47282251
## 31 -0.7364852 1.97239563 -2.90032314 -1.21123384
## 32 1.9349611 -1.34954321 -0.65151082 -0.69389106
## 33 -1.7133939 0.50711065 1.45551051 0.23951415
## 34 2.1715184 0.51867214 -0.79617459 0.72144670
## 35 1.8455912 -3.14503854 -1.90958277 -1.57486214
## 36 1.3688634 -1.62612600 0.62282006 -1.76868038
## 37 -4.0434633 0.11133255 2.48771689 -1.24088937
## 38 -0.7311405 3.03066799 -0.31670303 -0.36929862
## 39 -3.1914253 -1.05287718 0.19757744 -0.20595332
## 40 -0.8734081 -1.15060783 0.80069404 -0.38999635
## 41 1.6676747 1.99803996 1.73632335 -0.94882614
## 42 -0.5743410 1.78281860 -1.60020324 1.62781660
## 43 -2.2401511 -0.51933388 1.41947122 0.12199415
## 44 2.4639076 0.56769284 -0.71012766 -0.69260421
## 45 -2.8875348 -1.02635401 -3.38747512 -1.19886415
## 46 2.0194041 0.76712456 0.98934236 1.18517912
## 47 2.3475083 2.74265721 -1.51200217 0.47847727
```

I built a linear regression model with the first four principal components. Then, I used the `vif()` function to see if variance inflation was present.

```
crimepc = cbind(pc, crime[,16])
modelpc = lm(V5~., data = as.data.frame(crimepc))
summary(modelpc)
```

```
##
## Call:
## lm(formula = V5 ~ ., data = as.data.frame(crimepc))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -549.1  -237.8    20.3   132.5   771.7
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   857.267     44.333   19.337  <2e-16 ***
## PC1           38.034     18.529    2.053  0.0467 *
## PC2          -58.743     25.613   -2.294  0.0272 *
## PC3           30.977     31.304    0.990  0.3283
## PC4           -2.727     40.756   -0.067  0.9470
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```



```
## Residual standard error: 297.4 on 40 degrees of freedom
## Multiple R-squared:  0.2073, Adjusted R-squared:  0.128
## F-statistic: 2.614 on 4 and 40 DF,  p-value: 0.04944
```

```
vif(modelpc)
```

```
## PC1 PC2 PC3 PC4
##    1    1    1    1
```

We are transforming them back into the terms of the original variables to get the original coefficients.

In this code, the first matrix is the rotation matrix of the first four principal components. The rotation matrix is a matrix with the eigenvalues. This will tell us how each original variable contributes to each principal component. The second matrix is the model with the regression coefficients of the first four principal components, excluding the intercept. In other words, this is the linear combination of the original variables that correspond to my PCA regression model. I multiplied both matrices to get the model coefficients in the scaled variable space.

```
reverse_pca = pca$rotation[,1:4] %*% coef(modelpc)[-1]
reverse_pca
```

```
##           [,1]
## M          -9.6450155
## So         -2.1453350
## Ed           0.7252077
## Po1        30.1221351
## Po2        29.2561347
## LF         -2.9187865
## M.F        -28.2665945
## Pop        32.3978734
## NW          4.7906896
## U1        -21.3254829
## U2         -3.1140178
## Wealth    17.9911625
## Ineq     -11.7079122
## Prob     -20.9296090
## Time      27.5615149
```

pca\$center are the means of each original variable. *pca\$scale* are the standard deviations of each original variable. I stored into the variables “x_means” and “x_std”.

```
x_means = pca$center  
x_std = pca$scale
```

The next block of code is where I performed the transformation back to the original coefficients. I calculated the original coefficients by dividing the standard deviations from the model coefficients in the scaled variable space. This is the action of “unscaling”. I calculated the original intercept by getting the coefficients of the PCA model’s intercept and subtracting that by the sum of the mean of each scaled variable (x_means / x_std) and then multiplying that value by the regression coefficient vector in the scaled variable space.

```
coef_og = reverse_pca / x_std  
intercept = coef(modelpc)[1] - sum((x_means / x_std) * reverse_pca)
```

Here is the output of the original coefficients and intercept.

```
coef_og
```

```
##           [,1]  
## M      -7.540274e+00  
## So     -4.431683e+00  
## Ed       6.630320e-01  
## Po1      1.144515e+01  
## Po2      1.159890e+01  
## LF      -7.326307e+01  
## M.F     -1.050378e+01  
## Pop       9.526947e-01  
## NW       4.561531e-01  
## U1      -1.160321e+03  
## U2      -3.649660e+00  
## Wealth  1.929724e-02  
## Ineq    -2.922432e+00  
## Prob    -9.205938e+02  
## Time     3.829518e+00
```

```
intercept
```

```
## (Intercept)
##      1828.331
```

Finally, I used the testing points from the previous homework. In order to predict the crime rate, I used this formula: Intercept from original variable space + sum of (the coefficients from original variable space * the testing points).

```
test_point = data.frame(M = 14.0, So = 0, Ed = 10.0, Po1 = 12.0, Po2 = 15.5, LF = 0.640, M.F = 94.0, Po3 = 10.0, Po4 = 10.0, Po5 = 10.0, Po6 = 10.0, Po7 = 10.0, Po8 = 10.0, Po9 = 10.0, Po10 = 10.0, Po11 = 10.0, Po12 = 10.0, Po13 = 10.0, Po14 = 10.0, Po15 = 10.0)

pred_crime = intercept + sum(coef_og * unlist(test_point))

cat(paste('Predicted Crime Rate:', pred_crime))
```

```
## Predicted Crime Rate: 1118.8449247593
```

The predicted crime rate is 1118.

pcacenter are the means of each original variable. *pcascale* are the standard deviations of each original variable. *#sdev* = square root of eigenvalues *#rotation* = eigenvectors *#center* and *#scale* important for doing descaling. Gives you mean and std used to scale PR comp to scale data.

Now, I will redo the whole process but with the first 5 principal components instead.

```
pcal = prcomp(crime[,1:15], scale. = TRUE)
summary(pcal)
```

```
## Importance of components:
##              PC1      PC2      PC3      PC4      PC5      PC6      PC7
## Standard deviation  2.4196 1.7504 1.4322 1.10006 0.85240 0.7756 0.57949
## Proportion of Variance 0.3903 0.2043 0.1368 0.08068 0.04844 0.0401 0.02239
## Cumulative Proportion 0.3903 0.5946 0.7313 0.81199 0.86043 0.9005 0.92292
##              PC8      PC9      PC10     PC11     PC12     PC13     PC14
## Standard deviation  0.51908 0.49851 0.43510 0.42217 0.36402 0.26808 0.24737
## Proportion of Variance 0.01796 0.01657 0.01262 0.01188 0.00883 0.00479 0.00408
## Cumulative Proportion 0.94088 0.95745 0.97007 0.98195 0.99078 0.99558 0.99966
##              PC15
## Standard deviation  0.07191
## Proportion of Variance 0.00034
## Cumulative Proportion 1.00000
```

```
pca1$rotation[,1:5]
```

```
##           PC1           PC2           PC3           PC4           PC5
## M      -0.316254616  0.06305042  0.188947996 -0.08590892 -0.29854922
## So     -0.333187182 -0.17274096  0.033066443  0.23635812 -0.18753614
## Ed      0.333128872  0.23511610  0.067982291  0.08785745 -0.04829608
## Po1     0.306499286 -0.29432187  0.059628179  0.24644907 -0.30787047
## Po2     0.308676618 -0.28004655  0.058722722  0.27659132 -0.30255552
## LF      0.154517412  0.31717488  0.296573574 -0.23806501 -0.33408486
## M.F     0.064689141  0.42952225 -0.187822287 -0.11839142 -0.57035451
## Pop     0.109365270 -0.44953772  0.050640418 -0.09615164 -0.11686750
## NW     -0.304294463 -0.23263251  0.097529594  0.11826877 -0.41689121
## U1      0.022414151  0.03915007 -0.656623612 -0.16953306 -0.13364514
## U2     -0.005556309 -0.25276966 -0.582841039 -0.11135476 -0.09540101
## Wealth  0.383828141 -0.06408123 -0.003046747  0.10176965 -0.03783195
## Ineq    -0.374247145 -0.02940729  0.014667912 -0.12634975 -0.16170855
## Prob    -0.260948770  0.11704589 -0.086774938  0.52844557  0.08330971
## Time    -0.011946310 -0.34708314  0.194055820 -0.59258263  0.03442271
```

```
pc1 = pca1$x[,1:5]
```

```
crimepc1 = cbind(pc1, crime[,16])
modelpc1 = lm(V6~., data = as.data.frame(crimepc1))
summary(modelpc1)
```

```
##
## Call:
## lm(formula = V6 ~ ., data = as.data.frame(crimepc1))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -418.2 -189.1   -5.0  121.4  510.9
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   857.267     36.569   23.443 < 2e-16 ***
## PC1             38.034     15.284    2.488  0.01721 *
## PC2            -58.743     21.127   -2.780  0.00831 **
## PC3             30.977     25.822    1.200  0.23751
## PC4             -2.727     33.618   -0.081  0.93577
## PC5            -192.993     43.386   -4.448  7.01e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 245.3 on 39 degrees of freedom
## Multiple R-squared:  0.4741, Adjusted R-squared:  0.4067
## F-statistic: 7.031 on 5 and 39 DF,  p-value: 9.081e-05
```

```
vif(modelpc1)
```

```
## PC1 PC2 PC3 PC4 PC5
##   1   1   1   1   1
```

```

reverse_pca1 = pca1$rotation[,1:5] %*% coef(modelpc1)[-1]

x_means1 = pca1$center
x_std1 = pca1$scale

coef_og1 = reverse_pca1 / x_std1
intercept1= coef(modelpc1)[1] - sum((x_means1 / x_std1) * reverse_pca1)

test_point = data.frame(M = 14.0, So = 0, Ed = 10.0, Po1 = 12.0, Po2 = 15.5, LF = 0.640, M.F = 94.0, Pop
pred_crime1 = intercept1 + sum(coef_og1 * unlist(test_point))

cat(paste('Predicted Crime Rate:', pred_crime1))

```

```
## Predicted Crime Rate: 1351.81051548759
```

```

pca2 = prcomp(crime[,1:15], scale. = TRUE)
summary(pca2)

```

```
## Importance of components:
```

```

##          PC1      PC2      PC3      PC4      PC5      PC6      PC7
## Standard deviation  2.4196 1.7504 1.4322 1.10006 0.85240 0.7756 0.57949
## Proportion of Variance 0.3903 0.2043 0.1368 0.08068 0.04844 0.0401 0.02239
## Cumulative Proportion 0.3903 0.5946 0.7313 0.81199 0.86043 0.9005 0.92292
##          PC8      PC9      PC10     PC11     PC12     PC13     PC14
## Standard deviation  0.51908 0.49851 0.43510 0.42217 0.36402 0.26808 0.24737
## Proportion of Variance 0.01796 0.01657 0.01262 0.01188 0.00883 0.00479 0.00408
## Cumulative Proportion 0.94088 0.95745 0.97007 0.98195 0.99078 0.99558 0.99966
##          PC15
## Standard deviation  0.07191
## Proportion of Variance 0.00034
## Cumulative Proportion 1.00000

```

```
pca2$rotation[,1:6]
```

```

##          PC1      PC2      PC3      PC4      PC5
## M      -0.316254616  0.06305042  0.188947996 -0.08590892 -0.29854922
## So     -0.333187182 -0.17274096  0.033066443  0.23635812 -0.18753614
## Ed      0.333128872  0.23511610  0.067982291  0.08785745 -0.04829608
## Po1     0.306499286 -0.29432187  0.059628179  0.24644907 -0.30787047
## Po2     0.308676618 -0.28004655  0.058722722  0.27659132 -0.30255552
## LF      0.154517412  0.31717488  0.296573574 -0.23806501 -0.33408486
## M.F     0.064689141  0.42952225 -0.187822287 -0.11839142 -0.57035451
## Pop     0.109365270 -0.44953772  0.050640418 -0.09615164 -0.11686750
## NW     -0.304294463 -0.23263251  0.097529594  0.11826877 -0.41689121
## U1      0.022414151  0.03915007 -0.656623612 -0.16953306 -0.13364514
## U2     -0.005556309 -0.25276966 -0.582841039 -0.11135476 -0.09540101
## Wealth  0.383828141 -0.06408123 -0.003046747  0.10176965 -0.03783195
## Ineq    -0.374247145 -0.02940729  0.014667912 -0.12634975 -0.16170855
## Prob    -0.260948770  0.11704589 -0.086774938  0.52844557  0.08330971

```

```
## Time    -0.011946310 -0.34708314  0.194055820 -0.59258263  0.03442271
##          PC6
## M       -0.465568576
## So      -0.096852482
## Ed      -0.023720638
## Po1     -0.094575661
## Po2     -0.126105604
## LF       0.506557297
## M.F     -0.071667596
## Pop      0.534325805
## NW       0.041389686
## U1       0.003718574
## U2       0.059478556
## Wealth  -0.142460022
## Ineq     0.234609978
## Prob     0.314963943
## Time    -0.146326439
```

```
pc2 = pca2$x[,1:6]

crimepc2 = cbind(pc2, crime[,16])
modelpc2 = lm(V7~., data = as.data.frame(crimepc2))
summary(modelpc2)
```

```
##
## Call:
## lm(formula = V7 ~ ., data = as.data.frame(crimepc2))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -363.31 -155.17   8.34  134.62  462.58
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   857.267     35.921   23.866 < 2e-16 ***
## PC1           38.034     15.013    2.533  0.01554 *
## PC2          -58.743     20.753   -2.831  0.00738 **
## PC3           30.977     25.364    1.221  0.22949
## PC4           -2.727     33.022   -0.083  0.93463
## PC5          -192.993     42.617   -4.529 5.72e-05 ***
## PC6          -72.864     46.836   -1.556  0.12806
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 241 on 38 degrees of freedom
## Multiple R-squared:  0.5056, Adjusted R-squared:  0.4275
## F-statistic: 6.476 on 6 and 38 DF,  p-value: 9.12e-05
```

```
vif(modelpc2)
```

```
## PC1 PC2 PC3 PC4 PC5 PC6
##   1   1   1   1   1   1
```



```
summary(modelpc1)
```

```
##
## Call:
## lm(formula = V6 ~ ., data = as.data.frame(crimepc1))
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -418.2 -189.1   -5.0  121.4  510.9
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   857.267     36.569   23.443 < 2e-16 ***
## PC1           38.034     15.284    2.488  0.01721 *
## PC2          -58.743     21.127   -2.780  0.00831 **
## PC3           30.977     25.822    1.200  0.23751
## PC4           -2.727     33.618   -0.081  0.93577
## PC5          -192.993     43.386   -4.448  7.01e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 245.3 on 39 degrees of freedom
## Multiple R-squared:  0.4741, Adjusted R-squared:  0.4067
## F-statistic: 7.031 on 5 and 39 DF,  p-value: 9.081e-05
```

Let's compare the PCA model quality to our linear regression model from the previous homework. The linear regression model included the variables: Ed, Po1, Ineq, and Prob. Here is the summary of the model:

The regular model has a higher adjusted R-squared, lower RSE, and lower p-value, which are better model qualities. However, the variance is still unequal (heteroscedasticity). Let's compare it to the other VIF values from the PCA model.

```
vif(modelpc)
```

```
## PC1 PC2 PC3 PC4
##    1    1    1    1
```

The VIF values of the first four principal components are all 1. This is very good, and it proves that PCA removes multicollinearity. Therefore, all of the variances are equal and heteroscedasticity does not exist anymore. Even though the PCA regression had a lower R-squared, it removed multicollinearity, used fewer effective parameters, and reduced test error. Since our goal is to predict the crime rate, reducing test error means getting a more accurate prediction of the crime rate, which is what we should prioritize.

```
pca1 = prcomp(crime[,1:4], scale. = TRUE)
summary(pca1)
```

```
## Importance of components:
##              PC1      PC2      PC3      PC4
## Standard deviation   1.6001 0.8566 0.6509 0.53126
## Proportion of Variance 0.6401 0.1834 0.1059 0.07056
## Cumulative Proportion 0.6401 0.8235 0.9294 1.00000
```



```
pca2 = prcomp(crime[,1:5], scale. = TRUE)
summary(pca2)
```

```
## Importance of components:
```

##	PC1	PC2	PC3	PC4	PC5
## Standard deviation	1.780	1.0422	0.67224	0.53403	0.08159
## Proportion of Variance	0.634	0.2172	0.09038	0.05704	0.00133
## Cumulative Proportion	0.634	0.8512	0.94163	0.99867	1.00000

```
pca3 = prcomp(crime[,1:6], scale. = TRUE)
summary(pca3)
```

```
## Importance of components:
```

##	PC1	PC2	PC3	PC4	PC5	PC6
## Standard deviation	1.8122	1.2318	0.77217	0.55776	0.53370	0.08014
## Proportion of Variance	0.5474	0.2529	0.09938	0.05185	0.04747	0.00107
## Cumulative Proportion	0.5474	0.8002	0.89961	0.95146	0.99893	1.00000